

YipitData | Senior Software Engineer Assignment - Corporate

Objective

Create a full-stack application that allows our Brand and Retailer customers to receive a summary of key performance indicators (KPIs) like ASP (Average Sales Price), GMV (Sales \$), and Units Sold for different brands sold to retailers, enabling them to make decisions about what products/brands to stock and sell. The application should serve data through both a web frontend and an MCP (Model Context Protocol) server, enabling consumption by both human users and AI agents.

Problem Statement

YipitData produces monthly KPI estimates (e.g., ASP, GMV, and Units Sold) for many brands and retailers. For each (company, KPI), we publish:

- **Historical** monthly estimates, and
- **Month-to-Date (MTD)** estimates that reflect intra-month performance at a point in time.

Your app should make it easy for time-constrained Brand and Retailer stakeholders to:

1. Find a company and its KPIs, and
2. Understand trends at a glance (history vs MTD), with the ability to drill down and compare performance year-over-year (YOY) and month-over-month (MOM) when needed.

Example:

- Assume YipitData publishes estimates for **Trendy Shoe Brand (TSB)** KPIs each month within the **Footwear** sector.
- Some of these KPIs are: **GMV (Sales \$)**, **Units Sold**, and **ASP (Average Sales Price)**
- For each of these KPIs, YipitData will publish an estimate for the real value of these metrics on a monthly basis. We will also maintain a Month-to-Date (MTD) estimate that highlights what YipitData believes the current value to be at a point in time in the month, updated multiple times a day.

Goal

Design a full-stack web application that can display these estimates to customers. The system is intended to save them effort in digging through the various estimates and products available

in our portal. It is important to consider how customers will consume these and navigate the platform.

On the frontend, customers should be able to:

- Navigate available KPI estimates on the portal
- Display the 13-month history of KPI estimates and MTD estimate for a given company in chart format, with the ability to compare performance year-over-year (YOY) and month-over-month (MOM):
 - Filter by date range
 - Export current view
 - Show “last updated” / MTD “as-of” timestamps
- Receive in-app notifications when new KPI estimates are published while visiting the platform.
- Search for the sector, company or KPIs they care about and find the relevant pages.

In the backend, we should be able to:

- Return all the KPI estimates for a given company
- Publish new estimates for a given (company, KPI) pairing

In the MCP Server: The same data API that powers the frontend should also be exposed as an MCP (Model Context Protocol) server, enabling AI agents (e.g., Claude, ChatGPT) to query KPI data programmatically. Specifically:

- The MCP server should expose tools that allow an LLM to look up companies, retrieve KPI estimates, and query historical data
- Your README should include clear instructions on how to connect to and use the MCP server from an AI client (e.g., Claude Desktop, Cursor, or similar)
- Think about how an AI assistant would naturally want to interact with this data - what tools and parameters make sense from an LLM's perspective?

Securing the application: A simple authentication / authorization solution should be implemented but doesn't need an IDP or user store (hard coded credentials are fine, etc).

Example KPI Dataset to start with

You have been provided a sample dataset of KPI data to help inform your development.

Constraints/Assumptions

- Assume our customers are Brand and Retailer stakeholders who are time-constrained and need key insights fast about their product performance and market standing.

- Assume KPI datasets are already cleaned, processed, and available in a database (postgres)
- MTD snapshots updated multiple times a day.

Tech Stack

Frontend: Use any frontend language + framework you are comfortable with that spans Javascript/Typescript/React

Backend: Use **NodeJS** + whichever preferred framework for the backend. **Fastify** is recommended (Express, Nest, etc., are acceptable).

MCP: Use the same technology stack as the backend.

Expected Deliverables

- Working full-stack application codebase.
- A working MCP server that exposes the data API, with instructions on how to connect to it from an AI client
- A [README.md](#) with instructions on how to run the application. Include details about key architectural and design decisions you have made.
- A brief outline of future improvements you'd make to the application.

Evaluation Criteria

- Effectiveness of architecture in addressing key technical challenges
- Appropriateness and justification of technology choices given constraints and assumptions
- Reliability, performance and scalability of solution
- Robustness of observability, monitoring, and auditing plans
- Quality of MCP server implementation - tool design, discoverability, and how naturally an LLM can interact with the data

Live Review Session (60 minutes live coding)

- Be prepared to answer questions from the interviewer about your architecture and design decisions
- Be prepared to code follow-up features live based on interviewer questions
- Discuss any additional assumptions, trade-offs, or considerations you made in the process

Submission

Please submit any deliverables ahead of the scheduled live session. **You are highly recommended to use AI coding agents to assist with the exercise and produce the highest quality version you can.** You will still be expected to know all the implementation details, answer questions about the architecture / design decisions live, and code follow-up features in the live session.